Optimizing E-commerce Transaction Integrity: A Cost Sensitive Fraud Detection System

Zichao Zhang
Center for Data Science
New York University
New York, NY 10012
zz3491@nyu.edu

Shuang Wu
Center for Data Science
New York University
New York, NY 10012
sw5133@nyu.edu

Yexiang Tao
Center for Data Science
New York University
New York, NY 10012
yt3328@nyu.edu

Abstract

Online transaction fraud detection is an imbalanced and operationally limited classification task. The label imbalance is severe, the metadata are not always complete, and it is only feasible to examine the most likely cases of fraud. We address this challenge using machine learning methods on the IEEE-CIS Fraud Detection dataset. We compare a logistic regression classifier with tree classifiers under time-based validation. We further examine reprocessing techniques, feature engineering, and cost-sensitive tuning. Tree classifiers prove to be superior to the linear classifier, while missing data-aware features enhance ranking accuracy. However, difficulties persist in detecting fraud in sparse cases.

1 Introduction

1.1 Problem Motivation

Detection of fraud is one of the most important applications of machine learning. This is because of the possibility that not detecting fraud could lead to monetary losses, whereas excessive vigilance may lead to misclassifications of legitimate transactions. Furthermore, this problem cannot be addressed using only manual analysis since contemporary payment systems process vast amounts of transactions where suspicious patterns can be very subtle. And manual analysis can only be done on a limited sample. Moreover, there are problems such as extremely unbalanced classes, missing transaction data, and dynamic behavior of customers and criminals. Considering these factors, applying machine learning methods are natural to rank transactions by risk and help allocate limited review resources more effectively.

1.2 Why Fraud Detection Is a Difficult ML Problem

The challenge of detecting fraudulent activity lies not only in the fact that it is an unbalanced problem, but also in the complexity of the dataset and decision-making environment. As mentioned in the previous section, the occurrence of frauds is infrequent, many critical identity and device features are sparse, and the data distribution could evolve as the actions of legitimate customers and fraudulent individuals change. Moreover, the labels might be late, making it even more complicated to develop and evaluate algorithms than in the conventional benchmark problems for classification. At the same time, organizations do not treat these models as binary classifiers, using risk scores to sort transactions for manual review. As such, fraud detection presents a unique challenge for machine learning practitioners where each step in the pipeline, from preprocessing to evaluation, plays a crucial role.

Preprint.

1.3 Scope and Contributions

In this paper, we tackle transaction fraud detection as a ranking task using a temporally-aware formulation on the IEEE-CIS benchmark dataset. We contrast a baseline of carefully preprocessed logistic regression with tree-based classifiers, while investigating the impact of preprocessing decisions in a sparse and imbalanced setting. We proceed by studying missing-aware feature creation and evaluating cost-effective tuning under constrained review budgets, followed by analyzing failures to find areas of weakness for even strong models. In addition to comparing model effectiveness, our intention is to gain insight into the signals driving effective fraud detection and the shortcomings of existing approaches to it.

2 Related work

2.1 Fraud Detection as an Imbalanced and Time-Dependent Prediction Problem

Previous research uniformly considers fraud detection to be a challenging application of machine learning, due to the infrequency of frauds, imbalanced datasets, and dynamic transactional behavior. According to Dal Pozzolo et al., a successful fraud model should consider the temporal aspects of fraud detection and not simply treat it as a classification problem [1][2]. Additionally, Carcillo et al. suggest that purely supervised learning is restrictive when dealing with fraud detection, especially considering label delays and changes in patterns over time, thus requiring alternative solutions [3]. This paper shares this focus on considering fraud detection as a restricted prediction problem, but diverges in its empirical approach to evaluating the impact of different preprocessors, models, and evaluation methods.

2.2 Public Benchmarks and the IEEE-CIS Dataset

The datasets used in public fraud research are few in number, which makes IEEE-CIS a crucial benchmark in applying our methodology. Previous public studies have utilized datasets related to small instances of card-frauds such as those used in European cardholder benchmark study by Dal Pozzolo et al. [1][2]. In comparison to previous public studies, the IEEE-CIS Fraud Detection Competition offers a larger benchmark using real-world transaction instances in e-commerce setting, in which both transactions and identities are linked through TransactionID [4]. Such data structure proves useful when analyzing aspects such as missing values, sparsity in identities, heterogeneity of table features, and even distributional shift. Thus, our paper can be considered as another study on public benchmarks, however, we focus more on ranking-based evaluation, preprocessing ablation studies, and failure cases.

2.3 Tree-Based Models, Imbalanced Evaluation, and Interpretability

In terms of modeling, tree-boosting approaches are particularly well suited for the task of detecting fraud on sparse tabular data. While XGBoost was explicitly designed as a scalable system of tree boosting with sparsity-aware learning [5], LightGBM sought to be efficient and maintain a high predictive performance on large-scale data and spaces [6]. The above-mentioned characteristics are what makes tree-boosting models suitable for the IEEE-CIS dataset, characterized by high degrees of missingness and heterogeneity of features. On the evaluation side, Saito and Rehmsmeier demonstrate that in the case of imbalanced data, the precision–recall analysis tends to yield a better performance estimate compared to the ROC-analysis [7]. Finally, as fraud detection is a matter of considerable importance, we opt to use SHAP values for error analysis rather than consider the quality of predictions sufficient on their own account [8]. Hence, while our experiment may be regarded as consistent with previous tabular fraud-detection research in using tree-boosting as baseline models and proper evaluation techniques, it differs from these works by combining the comparison of such models with the previously mentioned elements into a coherent benchmark study.

3 Data

3.1 Dataset Overview

The primary data source for this project is the `IEEE-CIS Fraud Detection Dataset` from Kaggle, a recognized benchmark for evaluating e-commerce fraud detection models. This dataset is structured into two distinct tables—*Transaction* and *Identity*—which are interconnected via the `TransactionID` key. A critical characteristic of this data architecture is the partial overlap between the two sets; because not all transactions possess associated identity records, a significant portion of observations lacks identity-related features. Our modeling objective centers on the binary target variable, `isFraud`, with the goal of ranking transactions according to their calculated fraud probability.

The training data comprises a *Transaction* table with $590,540$ rows and 394 features, complemented by an *Identity* table containing $144,233$ rows across 41 columns. Upon executing a join operation on the `TransactionID` attribute, only 24.42% of the total transactions yield a corresponding match in the identity table. Furthermore, the dataset exhibits a pronounced class imbalance, as fraudulent transactions represent a mere 3.50% of the total volume. Interestingly, the data reveals a notable correlation between data availability and risk: transactions that include identity information demonstrate a significantly higher frequency of fraud compared to the general population.

3.2 Temporal Split and Sampling Strategy

For the early cost-sensitive tuning experiments, the full training dataset was first loaded into memory in chronological order. We then applied a time-based split, where the first 80% of observations were used as the training portion and the remaining 20% were held out as the validation set. This design preserves the temporal ordering of transactions and better reflects the deployment setting of fraud detection, where models are trained on past transactions and evaluated on later transactions.

To make the iterative cost-sensitive tuning process computationally feasible, we sampled only from the training portion of the data. Specifically, the training subset was reduced from 100,000 transactions to 80,000 transactions, while the held-out validation set was kept unchanged. As a result, model fitting and internal tuning were performed on temporally earlier training observations, and the final validation was evaluated on the full later-period validation set. We avoid random splitting because fraud patterns can change over time, and random splits may overstate performance by mixing future transaction behavior into the training process.

4 Methods

4.1 Data Preprocessing and Feature Engineering

We merge the transaction and identity tables by `TransactionID` using a left join. This keeps all transaction records and leaves identity fields missing when no identity record is available. Since identity coverage is incomplete, Given that the coverage of identity information is incomplete, we view this "missingness" itself as a valuable signal, rather than merely treating it as a simple data preprocessing issue.

The logistic regression baseline uses standard preprocessing: numerical imputation, categorical encoding, feature scaling, and class weighting. Tree-based models use the `tree_preprocessing_v2` framework. Within this framework, `v2_full` refers to the richest feature variant used in the final full-data XGBoost workflow.

Feature engineering includes `TransactionDT`-based time features, transaction-amount transformations, UID-style combinations, email-pair features, device-prefix tokens, row-level missingness summaries, count encoding for high-cardinality categorical variables, missing indicators, and group-level amount deviation features. These features capture timing, amount irregularities, repeated user-like patterns, category frequency, and missingness structure.

4.2 Baseline Model

To establish a quantitative performance floor, we employ a Logistic Regression (LR) model with standard preprocessing and class weighting to account for the rarity of fraudulent events. While

this linear benchmark offers exceptional interpretability and computational efficiency, its reliance on additive linear effects poses a significant constraint, as it inherently lacks the capacity to autonomously capture the high-order feature interactions, such as the synergistic relationship between device metadata and transaction velocity, that often characterize complex fraud patterns. Despite these structural limitations, the baseline provides critical insights into the linear separability of our engineered feature space, ensuring the benchmark represents a robust iteration of linear modeling rather than a naive implementation. Ultimately, the performance gap between this baseline and subsequent non-linear ensembles serves to isolate the incremental value derived from complex feature coupling and non-linear decision boundaries in the context of e-commerce integrity.

4.3 Tree-Based Model Comparison

We compare Random Forest, Extra Trees, HistGradientBoosting, CatBoost, LightGBM, and XGBoost. Random Forest and Extra Trees serve as bagging-based ensemble baselines. HistGradientBoosting, CatBoost, LightGBM, and XGBoost represent boosted tree methods for nonlinear tabular learning.

We compare both statistical and operational metrics, including ROC-AUC, Average Precision, top-K ranking metrics, and threshold-based metrics. Based on validation performance across these views, We select XGBoost as the main learner for the full-data refinement stage.

4.4 Model Refinement and Operational Evaluation

Our optimization phase centers on a full-data XGBoost workflow, utilizing a temporally-aware validation protocol where the model is trained on the initial 80% of chronological observations and evaluated on the subsequent 20%. This time-based split is critical to prevent future transaction patterns from leaking into the training process, thereby ensuring a realistic assessment of the model's performance in a deployment setting. To address the extreme class imbalance, we evaluated three architectural variants: a baseline reference, a hyperparameter-tuned model without explicit weighting, and a cost-sensitive configuration utilizing a tunable *scale_pos_weight* multiplier. This refinement targets the extraction of non-linear patterns—such as time-based features and identity-linked irregularities—from the high-dimensional feature set.

Given that missed fraud in e-commerce carries far greater financial consequences than false alarms, we move beyond a default 0.50 classification threshold. Instead, we calibrate the optimal operating point using a 10:1 false-negative to false-positive cost ratio, reflecting the assumption that missed fraud is substantially more detrimental than reviewing additional legitimate transactions. Performance is assessed through a dual lens of statistical robustness and operational utility: while ROC-AUC and Average Precision (PR-AUC) measure overall separability, Precision@TopK% and Recall@TopK% quantify the model's efficacy as a ranking engine within fixed audit budgets. This framework ensures that threshold selection is decoupled from ranking evaluation, allowing for a decision boundary that reflects both business priorities and finite inspection capacity.

5 Results

5.1 Overall Model Performance

Table 1 summarizes the main validation results on the time-based holdout set. The tuned XGBoost model without additional class weighting achieved the strongest overall validation performance, with a ROC-AUC of 0.9242 and a PR-AUC of 0.5839. Since fraudulent transactions are rare, PR-AUC and top-ranked review metrics are more informative than accuracy alone.

Table 1: Validation performance of the main XGBoost models.

Model	ROC-AUC	PR-AUC	Precision@Top3%	Recall@Top3%
`v2_full_reference`	0.9118	0.5531	0.5694	0.4966
`tuned_params_no_class_weight`	0.9242	0.5839	0.6010	0.5241
`tuned_cost_sensitive_xgb`	0.9215	0.5692	0.5861	0.5111

The selected model, `tuned_params_no_class_weight`, also achieved Precision@Top5% of 0.4338 and Recall@Top5% of 0.6304. This means that when reviewing the highest-risk 5% of validation transactions, the model captured approximately 63.0% of all fraud cases in the validation period.

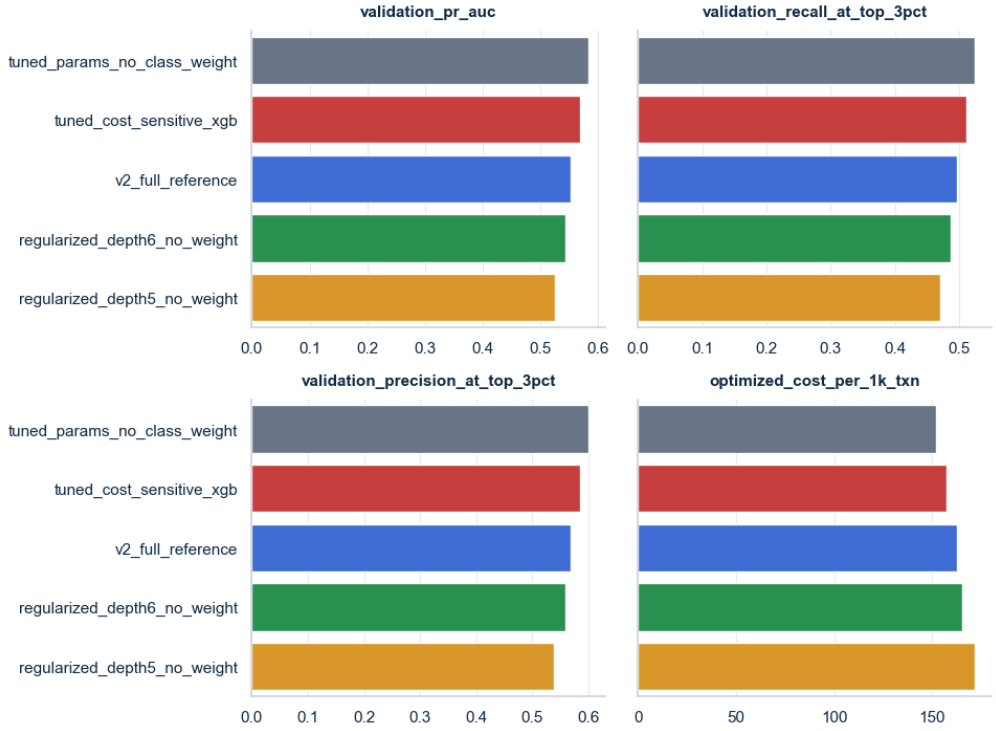

Figure 1: Validation performance comparison across candidate XGBoost models.

5.2 Business-Oriented Threshold Results

Because the model outputs fraud probabilities, a decision threshold is required to convert risk scores into fraud review decisions. We selected the operating threshold using a false-negative to false-positive cost ratio of 10:1. This setting reflects the assumption that missing a fraudulent transaction is substantially more costly than reviewing an additional legitimate transaction.

Under this cost setting, the selected model used an optimized threshold of 0.085. At this threshold, the model flagged 6.60% of validation transactions for review and achieved an expected cost of 151.76 per 1,000 transactions.

Table 2: Threshold-level operating results under FN:FP = 10:1.

Model	Threshold	False Positives	False Negatives	Cost / 1k
`v2_full_reference`	0.075	6688	1254	162.80
`tuned_params_no_class_weight`	0.085	5024	1290	151.76
`tuned_cost_sensitive_xgb`	0.415	5129	1343	157.14

Although the reference model produced slightly fewer false negatives at its optimized threshold, it required substantially more false positives and had a higher expected cost. The selected tuned model therefore provided the best cost-aware tradeoff under the 10:1 business assumption.

5.3 Regularization Check

To check whether the tuned model was overfitting, additional simplified XGBoost models were trained with lower tree depth and stronger regularization. These models reduced the train-validation

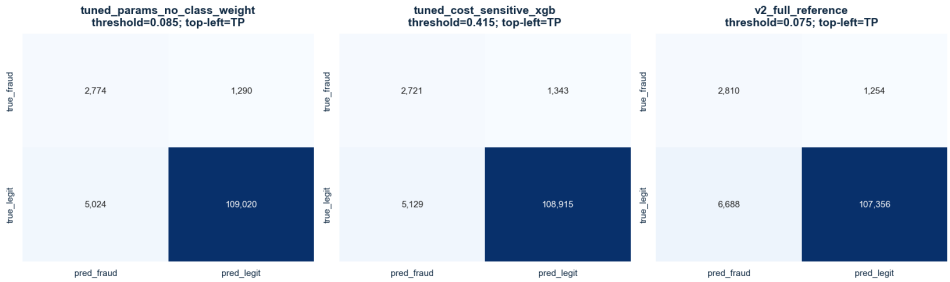

Figure 2: Confusion matrices at the optimized business threshold under the 10:1 cost setting.

PR-AUC gap, but they also reduced validation PR-AUC and increased expected cost. Therefore, the simpler models were not selected as the final model.

Table 3: Comparison with simplified regularized XGBoost variants.

Model	Validation PR-AUC	PR-AUC Gap	Cost / 1k
`tuned_params_no_class_weight`	0.5839	0.3018	151.76
`regularized_depth6_no_weight`	0.5434	0.2171	165.24
`regularized_depth5_no_weight`	0.5249	0.1913	171.62

These results suggest that the stronger tuned XGBoost model generalized better in terms of validation ranking performance and cost-aware decision quality, even though it showed a larger train-validation gap.

5.4 Summary

Overall, the tuned XGBoost model without additional class weighting achieved the best validation performance and the lowest expected cost under the selected 10:1 false-negative to false-positive cost ratio. The results also show that class weighting did not improve the final validation outcome in this setting, and that reducing model complexity led to underfitting rather than better operational performance.

6 Analysis

6.1 Threshold Tradeoff

Under a $10 : 1$ cost ratio, the optimized decision threshold was established at 0.085, a value substantially lower than the standard 0.5 cutoff. Such a shift is characteristic of fraud detection environments where flagging potential threats before they reach a high level of certainty is vital; while lowering the threshold inevitably expands the review queue, it simultaneously mitigates the risk of undetected fraudulent activity.

Rather than prioritizing the minimization of false negatives in isolation, the selected model focuses on reducing the overall expected cost. Although the reference model identified slightly more fraud cases, it incurred a higher cost per $1,000$ transactions due to an excessive volume of false positives. This distinction underscores that the final operating point serves to balance fraud capture against the operational review workload, rather than merely optimizing for a specific cell within the confusion matrix.

The dynamics of these competing priorities are illustrated in Figure 3, which depicts the interplay between recall, precision, and expected cost as the threshold fluctuates. By visualizing these curves, the inherent business tradeoff becomes apparent: as lower thresholds drive improvements in recall, they simultaneously trigger a decline in both precision and general review efficiency.

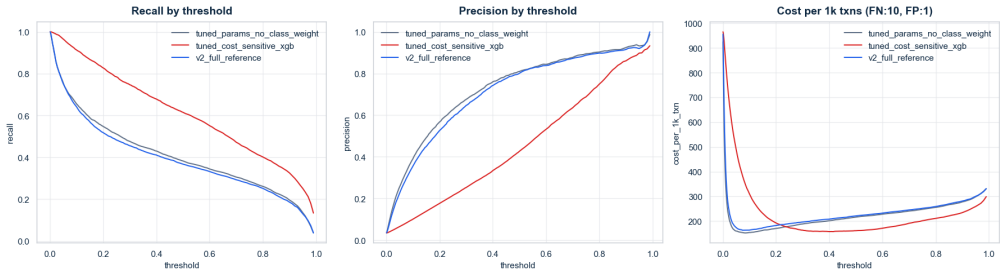

Figure 3: Recall, precision, and expected cost across classification thresholds under the 10:1 false-negative to false-positive cost setting.

6.2 Error Analysis

We analyzed false positives and false negatives at the selected threshold of 0.085. The final model produced 5,024 false positives and 1,290 false negatives on the validation set. The errors were not evenly distributed across transaction groups.

Table 4: Selected low-recall segments for missed fraud analysis.

Segment	Recall	Missed Fraud Rate
`ProductCD=W`	49.9%	50.1%
No identity information	50.5%	49.5%
Missing email information	51.8%	48.2%
`P_emaildomain=yahoo.com`	53.0%	47.0%
`TransactionAmt` bucket Q4	56.1%	43.9%

Table 4 shows that missed fraud was concentrated in lower-information or harder-to-separate segments. Transactions without identity information and transactions with missing email information were especially difficult. This likely limits the model's ability to use identity and email-based fraud signals.

We also compared each segment's share of errors with its share of validation rows. False negatives were overrepresented in segments such as `ProductCD=C`, `ProductCD=S`, missing address information, and `card4=discover`. False positives were overrepresented in high-risk-looking segments such as `ProductCD=C`, `ProductCD=H`, missing address information, and credit-card transactions.

This explains part of the threshold tradeoff. Some transaction groups resemble fraud strongly enough to create false alarms, while other groups lack enough signal for the model to detect all fraud cases.

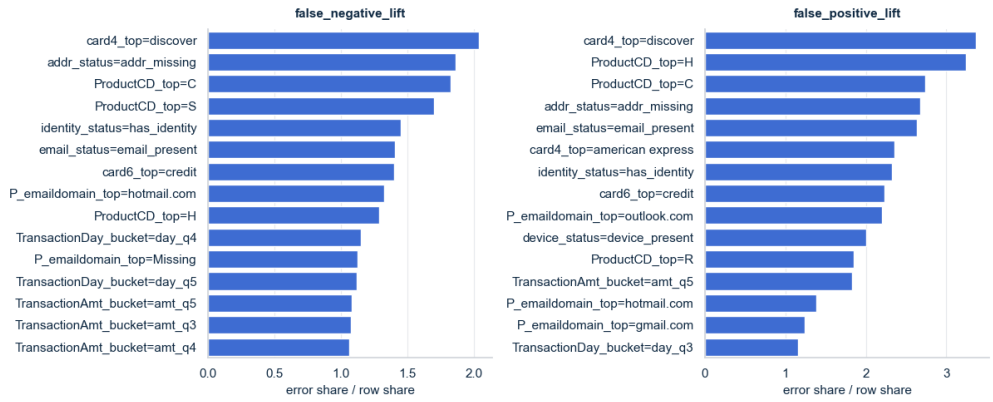

Figure 4: Overrepresented false-negative and false-positive segments at the optimized threshold.

6.3 SHAP Interpretation

SHAP analysis showed that the three main XGBoost models used similar top features. The selected tuned model and the reference model shared 17 of their top 20 SHAP features. The selected tuned model and the cost-sensitive model shared 15 of their top 20 SHAP features.

Table 5: Top SHAP feature overlap across main models.

Model Pair	Shared Top-20 Features
Selected tuned model vs. reference model	17
Selected tuned model vs. cost-sensitive model	15
Reference model vs. cost-sensitive model	15

This suggests that tuning improved performance mainly by changing feature weights and interactions, not by learning a completely different decision rule. Important features included anonymized transaction behavior variables such as `C13`, `V258`, `C14`, and `C1`, along with engineered count features such as `card6__count`.

The engineered amount-deviation feature `TransactionAmt_diff_mean_addr1` also appeared among important features. This supports the value of comparing a transaction amount against typical amounts within the same address group.

Figure 5 shows both feature importance and direction. Points on the right push predictions toward fraud, while points on the left push predictions toward legitimate. The color indicates whether the feature value is high or low. Clear color separation across the horizontal axis indicates a more consistent directional effect.

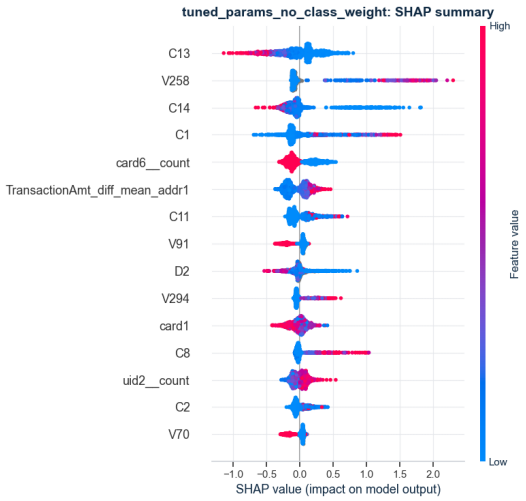

Figure 5: SHAP summary plot for the selected tuned XGBoost model. Points to the right increase predicted fraud risk, while points to the left decrease it.

References

[1] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 2018.

[2] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Learned Lessons in Credit Card Fraud Detection from a Practitioner Perspective. *Expert Systems with Applications*, 2014.

[3] Fabrizio Carcillo, Yann-Aël Le Borgne, Olivier Caelen, Yacine Kessaci, and Frédéric Oblé. Combining Unsupervised and Supervised Learning in Credit Card Fraud Detection. *Information Sciences*, 2021.

[4] Kaggle / IEEE-CIS. IEEE-CIS Fraud Detection. Kaggle Competition Page, 2019.

[5] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

[6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] Takaya Saito and Marc Rehmsmeier. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 2015.

[8] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.